

سوال 2:

(1)

- از بین رفتن رفتار Short Circuit: در ++C استاندارد، عملگرهای && و || دارای خاصیت Short circuit Evaluation هستند. یعنی:
 - در A && B، اگر A غلط باشد، B اصلا اجرا نمی شود.
 - در A || B، اگر A درست باشد، B اصلا اجرا نمی شود.
- اما وقتی شما این عملگرها را overload می کنیم، آن ها تبدیل به فراخوانی تابع معمولی می شوند (مثل operator&&(A, B) در فراخوانی تابع، تمامی آرگومان ها قبل از ورود به تابع بررسی می شوند.
- در نتیجه اگر بخش دوم شرط (B) شامل کدی باشد که نباید اجرا شود (مثلا چک کردن ptr->val وقتی ptr نال است، یا دارای Side Effect مثل تغییر متغیر باشد)، برنامه ممکنه کرش کند یا رفتار ناخواسته نشان بده چون B همیشه اجرا می شود.

(2)

- مشکل: اگر کاربر بنویسد if (a != b)، کد کامپایل نمی شود. کامپایلر به صورت خودکار != را معکوس == نمیداند.
- راه حل: همیشه هر دو را با هم پیاده سازی می کنیم:

```
1  bool operator!=(const MyClass &other) const
2  {
3      return !(*this == other);
4  }
```

- خطر ناسازگاری: اگر == بگوید دو شی برابرند، != هم بگوید برابر نیستند (هر دو true برگردانند)، منطق برنامه به هم می ریزد.

(3)

تحلیل وضعیت

کلاس شما operator= (عملگر انتساب) را overload کرده، اما Copy Constructor ندارد.

- آیا کامپایلر خطا می دهد؟
 - خیر کامپایلر یک Default Copy Constructor می سازد.
- چه خطری دارد؟
 - سازنده پیش فرض فقط یک کپی سطحی (Shallow Copy) انجام می دهد. اگر کلاس شما دارای اشاره گر باشد، شی جدید به همان خانه حافظه شی قبلی اشاره می کند.
- خطای Runtime:
 - این موضوع باعث Double Free می شود.
 - شی A ساخته می شود و حافظه می گیرد.
 - شی B از روی A ساخته می شود حالا هر دو به یک حافظه اشاره میکنند
 - وقتی B از بین می رود Destructor فعال میشود
 - وقتی A از بین می رود، سعی می کند همان حافظه آزاد شده را دوباره آزاد کند نتیجه اش میشود کرش کردن

(4)

- Overload کردن باید فقط زمانی انجام شود که عملیات معنای واضحی داشته باشد.
- خوانایی کد: اگر بنویسید `Employee a, b; a + b`، باعث ابهام برنامه نویسان دیگر می شود چون معلوم نیست دقیقا منظور از `+` چیست.
 - گمراهی: استفاده از توابع با نام مشخص (مثل `addSalary` یا `mergeNames`) بسیار خوانا تر و امن تر از استفاده از `+` یا `*` است.
-

(5)

چرا باید نسخه `const` و `non const` وجود داشته باشد؟

1. نسخه `non const`: برای زمانی که می خواهیم مقدار آرایه را تغییر دهیم.
`int &operator[] (int index);`
2. نسخه `const`: برای زمانی که می خواهیم `const` است (مثلا به صورت `&const Array` به تابع پاس داده شده) و فقط می خواهیم مقدار را بخوانیم.
`const int &operator[] (int index) const;`

اگر فقط `non const` باشد چه می شود؟

اگر یک شی `const` داشته باشیم (مثلا `void print(const Array& a)`) و بخواهیم `a[0]` را بخوانیم، کامپایلر خطا می دهد. زیرا `operator[]` معمولی تضمین نمی کند که شی را تغییر ندهد، پس روی شی `const` قابل صدا زدن نیست.

برگرداندن مقدار بهتر است یا مرجع؟ چرا؟

- برگرداندن مرجع بهتر است.
1. امکان تغییر: اگر `by value` برگردانیم، `arr[0] = 5` کار نمی کند، چون ما داریم عدد 5 را داخل یک کپی موقت می ریزیم که بلافاصله از بین می رود. برای اینکه `arr[i]` بتواند سمت چپ مساوی قرار بگیرد، باید رفرنس به خانه حافظه واقعی برگردانده شود.
 2. کارایی: اگر نوع داده داخل آرایه بزرگ باشد (مثلا آرایه ای از اشیا سنگین) برگرداندن با مقدار باعث کپی شدن بیهوده اطلاعات می شود.